

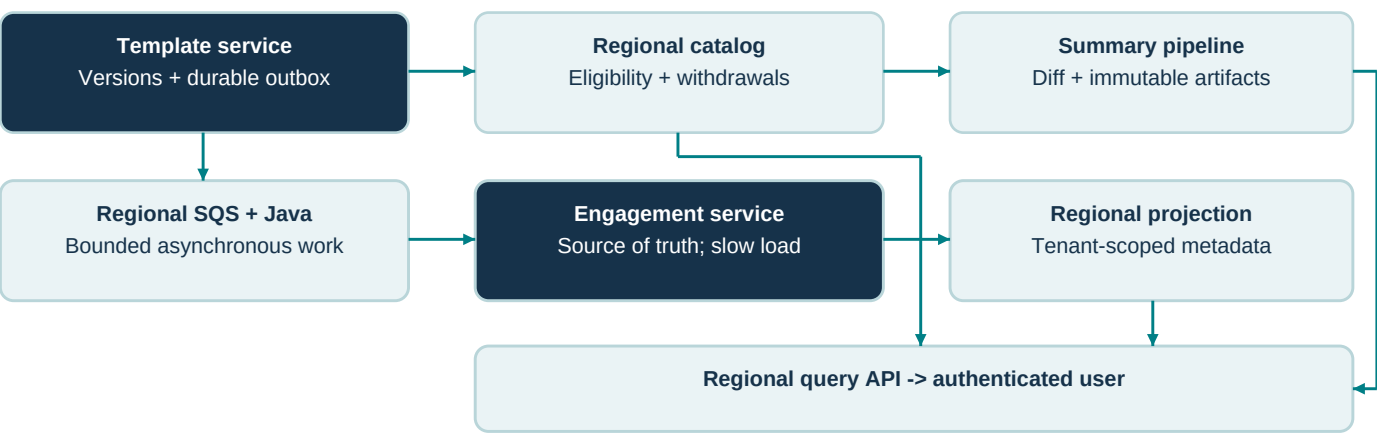
Template Update Intelligence

01 / System boundaries and the read path

JHONNATAN DUQUE RAMOS
jonathan.duque.ramos@gmail.com | +573007407336

Decision. Maintain a regional, event-driven metadata projection and resolve pending updates against a small version catalog at read time. Keep minute-long engagement processing asynchronous. A template publication must not require rewriting every engagement before users can see an update.

Architecture and ownership



Arrows show logical data flow. Publication events reach each region; only non-firm-specific template content and summaries may be shared across regions. The query API reads the catalog, projection and summary artifacts.

Template service owns immutable versions, parent relationships, market/channel compatibility, withdrawal state and a monotonic catalog revision. A published catalog revision is installed atomically in each region. **Engagement service owns** the actual applied version and accept/decline decisions; the projection is rebuildable, never authoritative.

Regional projection: firm ID, engagement ID, template ID, market/branch, applied version, declined offer ID, source revision, metadata status and timestamp. Index by (firm, engagement) for paginated lists and (template, market, engagement) for fan-out. Keep a separate immutable decision log. A creation version alone is insufficient after an update.

Query: fetch one authorized page of metadata, resolve eligible targets in a batch, then retrieve cached summaries by source/target pair. A catalog change makes updates visible without per-engagement fan-out writes. Unknown metadata is explicitly shown as “not yet verified”; missing summaries appear as “preparing”.

AWS mapping: regional PostgreSQL/RDS metadata and job state; SQS queues; Java containers on ECS; S3 for versioned summary evidence. Reuse existing services where suitable. Tenant authorization applies to every query, job and artifact access. EU/Canada engagement metadata, decisions, queues, logs and backups remain in-region; template replication is shared.

Correctness before freshness

02 / Version semantics, migration and safe evolution

Explicit offer semantics

Versions form a directed acyclic graph, not an ordered number sequence. Product supplies an explicit compatibility policy and preferred target per market/channel. If two branches are incomparable and no policy selects one, surface ambiguity instead of guessing. A withdrawn target cannot receive new accepts; retain its content for historical evidence.

An **offer** records template, source version, target version and eligibility-policy revision. Rejecting v5 suppresses that offer, not its changes forever. If the applied version is v3, a later v7 offer summarizes **v3 to v7**, including relevant v5 changes. A newer eligible target supersedes an unanswered older offer. This policy is an assumption to confirm with Product.

Accept/decline commands carry the offer ID and expected engagement revision. The authoritative service checks both current eligibility and source revision, then commits a decision and event. A stale offer is rejected and refreshed.

Acceptance records intent; the applied version advances only after confirmed application by the existing workflow, which is outside this design.

Events that tolerate failure and races

Use a transactional outbox where supported to atomically persist a source change and its event [1]. Otherwise require equivalent change capture or a durable source journal. Events contain unique IDs and monotonic per-aggregate revisions. Consumers deduplicate, reject older revisions and detect gaps. Full-state snapshots simplify recovery; deltas require ordered replay. Standard SQS delivery can repeat [2].

Reconcile publication revisions against the small authoritative catalog. Reconcile engagement metadata using durable source revisions/events; schedule slow reloads for unresolved discrepancies. Never claim exactly-once remote execution without cooperation from the downstream service. Historical decisions and viewed summary IDs remain append-only.

Online migration without a maintenance window

1. Capture first. Deploy hooks for creation, decisions and successful application before scanning; retain events durably from a migration checkpoint. Assume a tenant-scoped inventory can enumerate existing engagement IDs without opening them; validate this dependency early.

2. Backfill gradually. Load each legacy engagement through the existing one-minute operation under a shared capacity limit. Capture metadata and its authoritative revision consistently. Upsert only if that revision is newer; replay retained events so a delayed scan cannot overwrite a recent decision. If no coherent revision can be exposed, this is a correctness blocker to resolve with the owning team.

3. Roll out safely. Checkpoint inventory pages, retry failures and compare sampled projection results with source loads. Enable per-tenant reads behind a feature flag after adequate coverage. Capture metadata opportunistically when users open files. Keep uncatalogued legacy files in an explicit unknown state. Rollback disables new reads while event capture continues.

The requirement I would challenge

Seconds-level freshness for every legacy engagement from day one is incompatible with the hard load constraint.

Offer seconds-level catalog freshness for indexed engagements and a measured migration window for the rest. Full fan-out completion has its own capacity-based objective. Also negotiate whether a pending indicator may precede a validated natural-language summary.

Highest-risk error: showing a plausible but wrong offer after a race, withdrawal or branch change. Protect it with source revisions, explicit eligibility rules, transactional decisions and replay tests. A fast incorrect indicator is worse than a visible unknown state.

Bounded publish fan-out

03 / Required Java component and production controls

Scope. The required Java worker still invokes the one-minute downstream operation for each selected engagement; it refreshes/reconciles metadata asynchronously. The fast read path does not wait for it. This document specifies the implementation contract; the Java source and short README are separate required deliverables.

Durable work with a shared capacity budget



Expired lease / transient failure -> delayed retry; terminal failure -> DLQ

Fan-out coordinator: persist publication ID and enumeration cursor; page through the regional index, insert unique tasks and outgoing messages transactionally, then advance the cursor. Resume after crashes. New engagements obtain metadata on creation and use current catalog eligibility, so they do not depend on an earlier scan. Coalesce obsolete targets only after re-evaluating current eligibility.

Interfaces: EngagementEnumerator.page(cursor), TaskStore.claim/complete, CapacityLimiter.acquire/renew/release, EngagementGateway.loadMetadata(id, operationKey), and Catalog.resolveTarget(metadata). Inject a clock and retry policy for deterministic tests. Task identity is (firm, engagement, target, operation type); retries preserve that identity.

Execution: claim a task using a conditional update and fencing token, acquire a shared regional/downstream permit, validate the target, call the gateway, persist the result with a source-revision check, mark completion, then acknowledge the message. The concurrency ceiling applies across replicas and includes migration traffic. Fair tenant scheduling and per-tenant caps prevent a 40,000-file firm from starving smaller firms.

Uncertain outcomes: use bounded timeouts, exponential backoff with jitter and a retry budget. Renew queue visibility and ownership while work runs. An expired lease does not prove the remote call stopped: retain uncertain capacity until cancellation/status is confirmed or a contractual maximum runtime passes. Side-effecting calls need downstream idempotency or operation-status lookup; local deduplication alone cannot guarantee safety.

Failure recovery and validation

Retry throttling and transient failures; route permanent or exhausted failures to a dead-letter queue with an operator-visible reason and safe redrive. Circuit breaking stops new calls during an outage. Resume from durable cursors and task state. Revalidate after withdrawal and before persisting a result; never advance an applied version merely because a worker completed.

Essential tests: duplicate/out-of-order events; crash after remote success but before commit; lease expiry during an active call; multi-replica capacity enforcement; tenant fairness; withdrawal mid-flight; backfill racing with accept/decline. Use a slow/failing fake gateway and verify invariants, not just happy-path output.

Proposed service objectives and signals

Targets to validate: 99% of catalog changes queryable in-region within 10 seconds; indexed list API p95 under 300 ms at agreed load; 99.9% monthly API availability. Migration coverage, summary readiness and fan-out completion are separate metrics, not covered by the 10-second target. Front-end polling or push delivery must fit any end-to-end freshness promise.

Monitor oldest event/task age, revision gaps, metadata coverage, retry/DLQ rates, active and uncertain permits, downstream throttling, summary validation failures and cost per publication. Alert on growing backlog and capacity saturation. On catalog lag, expose freshness; fail closed for decisions whose eligibility cannot be checked. Recovery uses replay, regional backups and reconciliation.

Scale, cost and trustworthy AI

04 / Quantitative tradeoffs, summary evidence and limits

Capacity arithmetic and cost envelope

Quantity / assumption	Calculation and implication
Legacy backfill	800,000 x 60 s = 48M call-seconds. At C=100 concurrent calls: 5.56 days minimum, before retries and reserved user capacity.
Average publication (uniform mix only)	800,000 / 40 = 20,000 files. At C=100: 20,000 x 60 / 100 = 3.33 hours. Worst case: 800,000 files; size each actual product.
Steady weekly refresh	40 publications/week; roughly 800,000 calls/week if every file is processed once. Average concurrency = 800,000 / 10,080 = 79.4. C=100 has little spare capacity for peaks/backfill.
Monthly work and storage	4.33 weeks/month gives 3.46M calls and 57,733 downstream call-hours. At an assumed 1 KB/metadata row: 0.8 GB raw, before indexes, replicas and history.

Budget model: monthly cost = regional fixed infrastructure + 3.46M x incremental cost/call + queue requests x unit price + storage/IO + summary inference + observability. At illustrative effective costs of \$0.0001 / \$0.001 / \$0.01 per call, downstream work alone is \$346 / \$3,464 / \$34,640. These are sensitivity inputs, not AWS quotes. Retries and migration add cost; call-hours are not CPU-hours.

The brief leaves **\$X undefined**; budget compliance cannot be claimed. Measure marginal downstream cost and reserved capacity with the owning team, price the chosen regions, then agree X. Avoid recurring full loads once projection correctness is proven if Product permits; retain the required worker for controlled reconciliation. Reducing reloads is the dominant optimization, not adding replicas.

Generate once per version pair; preserve what was shown

Use the reliable JSON diff as evidence. Cache by template, source/target content hashes, locale, summarizer version and policy. Generate deterministic change categories first; an optional LLM rewrites them into plain language. Only template content enters inference. Treat template text as untrusted data, constrain the output schema and link every claim to diff paths. Reuse outputs across firms; do not infer over client workpapers.

Evaluate against a curated expert-reviewed set: factual support, important-change coverage, numerical fidelity, language clarity and behavior on deletions/withdrawals. Require no unsupported material claims and complete coverage of designated critical changes before release. On failure, serve a validated deterministic summary or show pending status; never invent implications or audit conclusions.

Store immutable diff, exact summary text, input hashes, model identifier, prompt/configuration, evaluator results and generation time. A regional view/decision record binds the user, offer and artifact ID. Retain the March artifact for a November review; regeneration is a new artifact, not a replacement. Agree retention and access policy with compliance. Inference cost = cache misses x tokens x contracted token rates; active version pairs, not engagement count, determine misses.

Deliberate limits and AI accountability

Excluded: applying template content, a full UI, autonomous audit decisions and global replication of firm data. Keep branch policy explicit rather than building a generic workflow engine. AI assisted this proposal's structure and drafting; correctness, cost and domain policies require human review. The candidate should record actual corrections and be ready to defend the design; no implementation, load test or evaluation result is claimed here.

References: [1] [AWS: Transactional outbox pattern](#). [2] [AWS: SQS at-least-once delivery](#). Requirements: supplied Caseware take-home brief.